

1. O que é uma REST API?

Imagine que temos um sistema de vendas e queremos permitir que outros sistemas consultem os produtos.

Um cliente, por exemplo um aplicativo React, poderia fazer:

```
GET http://localhost:8080/produtos
```

A API recebe a solicitação, consulta os dados e devolve uma resposta, normalmente em JSON:

```
</> JSON  
  
[  
  {  
    "id": 1,  
    "nome": "Notebook",  
    "preco": 3500.00  
  },  
  {  
    "id": 2,  
    "nome": "Mouse",  
    "preco": 80.00  
  }  
]
```

A ideia central é: **REST** é um estilo arquitetural para construir serviços que disponibilizam recursos através do protocolo **HTTP**.

Nesse exemplo, **Produto** é um recurso.

2. Recursos e URLs

Em REST, normalmente usamos **substantivos** nas URLs.

Por exemplo:

```
/produtos  
/clientes  
/pedidos  
/usuarios
```

E utilizamos os métodos HTTP para indicar a operação.

Método	Objetivo	Exemplo
GET	Consultar	GET /produtos
GET	Consultar um	GET /produtos/10
POST	Criar	POST /produtos
PUT	Alterar completamente	PUT /produtos/10
PATCH	Alterar parcialmente	PATCH /produtos/10
DELETE	Excluir	DELETE /produtos/10

Uma maneira simples de memorizar:

```

GET      → buscar
POST     → criar
PUT      → substituir/alterar
PATCH   → alterar parcialmente
DELETE   → excluir

```

3. O que significa API?

API significa:

Application Programming Interface

Podemos imaginar uma API como uma "porta de comunicação" entre sistemas.

Por exemplo:

```

React
|
| HTTP
↓
REST API
|
| JDBC/JPA
↓
Banco de Dados

```

O React não precisa saber como o banco de dados funciona. Ele apenas conversa com a API.

4. Arquitetura em camadas

Para uma aplicação Java com Spring Boot, uma arquitetura simples e muito utilizada é:

```

Controller
↓
Service
↓
Repository

```

↓
Banco de Dados

Cada camada possui uma responsabilidade.

Controller: Recebe as requisições HTTP.

```
GET /produtos
POST /produtos
DELETE /produtos/10
```

Service: Contém as regras de negócio.

Por exemplo:

```
Produto não pode ter preço negativo.
Produto não pode ser vendido sem estoque.
```

Repository: Responsável pelo acesso aos dados.

```
SELECT
INSERT
UPDATE
DELETE
```

Com Spring Data JPA, boa parte disso é gerada automaticamente.

5. Estrutura do projeto

Vamos criar uma API de produtos.

Uma estrutura simples:

```
src/main/java/com/exemplo/api
├── controller
│   └── ProdutoController.java
├── service
│   └── ProdutoService.java
├── repository
│   └── ProdutoRepository.java
├── model
│   └── Produto.java
└── ApiApplication.java
```

O fluxo será:

```
Cliente
↓
Controller
↓
Service
↓
```

Repository

↓

Banco

6. Criando a entidade Produto

```
package com.exemplo.api.model;

import jakarta.persistence.*;

@Entity
@Table(name = "produtos")
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;

    private Double preco;

    public Produto() {
    }

    public Produto(Long id, String nome, Double preco) {
        this.id = id;
        this.nome = nome;
        this.preco = preco;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public Double getPreco() {
        return preco;
    }

    public void setPreco(Double preco) {
        this.preco = preco;
    }
}
```

Aqui estamos dizendo ao JPA:

A classe `Produto` representa uma tabela no banco de dados.

7. Criando o Repository

```
package com.exemplo.api.repository;

import com.exemplo.api.model.Produto;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProdutoRepository
    extends JpaRepository<Produto, Long> {

}
```

E aqui temos uma grande facilidade do Spring Data JPA.

Não precisamos implementar manualmente:

```
SELECT * FROM produtos;
```

O `JpaRepository` já fornece métodos como:

```
findAll()
findById()
save()
deleteById()
existsById()
```

8. Criando o Service

Agora criamos a camada responsável pela lógica de negócio.

```
package com.exemplo.api.service;

import com.exemplo.api.model.Produto;
import com.exemplo.api.repository.ProdutoRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class ProdutoService {

    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }

    public List<Produto> listar() {
        return repository.findAll();
    }

    public Produto buscarPorId(Long id) {
        return repository.findById(id)
            .orElseThrow(() ->
```

```

        new RuntimeException("Produto não
encontrado"));
    }

    public Produto criar(Produto produto) {

        if (produto.getPreco() == null ||
            produto.getPreco() < 0) {

            throw new IllegalArgumentException(
                "Preço deve ser maior ou igual a zero");
        }

        return repository.save(produto);
    }

    public void excluir(Long id) {

        if (!repository.existsById(id)) {
            throw new RuntimeException(
                "Produto não encontrado");
        }

        repository.deleteById(id);
    }
}

```

Observe uma coisa importante.

O Controller **não está conversando diretamente com o Repository**.

Temos:

```

Controller
  ↓
Service
  ↓
Repository

```

Isso ajuda a separar responsabilidades.

9. Criando o Controller

Agora chegamos à parte REST.

```

package com.exemplo.api.controller;

import com.exemplo.api.model.Produto;
import com.exemplo.api.service.ProdutoService;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

```

```

@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    private final ProdutoService service;

    public ProdutoController(ProdutoService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<List<Produto>> listar() {

        return ResponseEntity.ok(
            service.listar()
        );
    }

    @GetMapping("/{id}")
    public ResponseEntity<Produto> buscar(
        @PathVariable Long id) {

        return ResponseEntity.ok(
            service.buscarPorId(id)
        );
    }

    @PostMapping
    public ResponseEntity<Produto> criar(
        @RequestBody Produto produto) {

        Produto novoProduto =
            service.criar(produto);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(novoProduto);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> excluir(
        @PathVariable Long id) {

        service.excluir(id);

        return ResponseEntity.noContent().build();
    }
}

```

Agora temos uma REST API funcionando conceitualmente.

10. Entendendo os códigos HTTP

Essa é uma parte **muito importante** para quem está começando com REST.

Não devemos retornar sempre 200 OK.

O código HTTP deve representar o resultado da operação.

GET bem-sucedido

200 OK

Exemplo:

GET /produtos

Resposta:

200 OK

GET de recurso inexistente

Imagine:

GET /produtos/999

Se o produto não existe, o correto é:

404 NOT FOUND

Não devemos retornar:

200 OK

com uma mensagem dizendo:

```
{
  "erro": "Produto não encontrado"
}
```

O status HTTP deve representar o resultado.

POST criando recurso

Quando fazemos:

POST /produtos

e o produto é criado, o status recomendado é:

201 CREATED

Por isso usamos:

```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .body(novoProduto);
```

DELETE

Se o produto foi excluído:

204 NO CONTENT

Por isso:

```
return ResponseEntity.noContent().build();
```

11. Resumo dos principais códigos

Código	Significado	Quando utilizar
200	OK	Consulta/alteração realizada
201	Created	Recurso criado
204	No Content	Operação realizada sem conteúdo para retornar
400	Bad Request	Requisição inválida
401	Unauthorized	Usuário não autenticado
403	Forbidden	Usuário autenticado, mas sem permissão
404	Not Found	Recurso não encontrado
409	Conflict	Conflito, como cadastro duplicado
422	Unprocessable Content	Dados semanticamente inválidos
500	Internal Server Error	Erro inesperado no servidor

12. Mas temos um problema no código

No nosso exemplo:

```
@GetMapping("/{id}")
public ResponseEntity<Produto> buscar(
    @PathVariable Long id) {

    return ResponseEntity.ok(
        service.buscarPorId(id)
    );
}
```

O Service lança:

```
throw new RuntimeException("Produto não encontrado");
```

Isso provavelmente produziria:

```
500 Internal Server Error
```

Mas sabemos que o correto seria:

```
404 Not Found
```

Então precisamos tratar as exceções.

13. Criando uma exceção específica

Vamos criar:

```
package com.exemplo.api.exception;

public class ProdutoNaoEncontradoException
    extends RuntimeException {

    public ProdutoNaoEncontradoException(String mensagem) {
        super(mensagem);
    }
}
```

No Service:

```
public Produto buscarPorId(Long id) {

    return repository.findById(id)
        .orElseThrow(() ->
            new ProdutoNaoEncontradoException(
                "Produto não encontrado"
            )
        );
}
```

Agora temos uma exceção específica.

14. Tratando a exceção globalmente

Podemos criar:

```
package com.exemplo.api.exception;

import org.springframework.http.HttpStatus;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProdutoNaoEncontradoException.class)
    public ResponseEntity<String> tratarProdutoNaoEncontrado(
        ProdutoNaoEncontradoException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

Agora:

GET /produtos/999

produzirá:

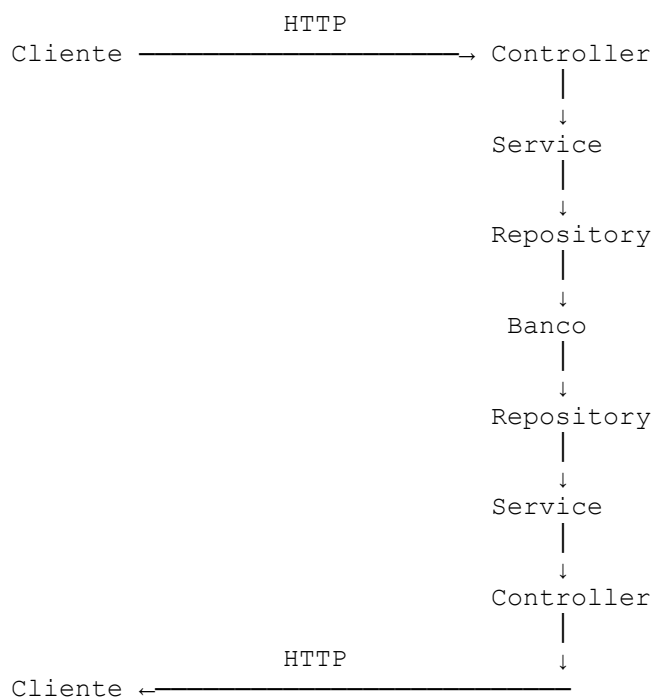
404 NOT FOUND

15. O fluxo completo

Quando o cliente faz:

GET /produtos/10

acontece:



Esse é um dos conceitos mais importantes para entender Spring Boot.

16. Exemplo de POST

O cliente poderia enviar:

```
POST /produtos
Content-Type: application/json
```

Com:

```
{
  "nome": "Teclado",
  "preco": 150.00
}
```

O Controller recebe:

```
@PostMapping
public ResponseEntity<Produto> criar(
    @RequestBody Produto produto) {
```

O Spring transforma automaticamente o JSON em um objeto Java:

```
Produto produto
```

Depois:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Banco
```

Se tudo der certo:

```
HTTP/1.1 201 Created
```

E podemos retornar:

```
{
  "id": 3,
  "nome": "Teclado",
  "preco": 150.0
}
```